

Module 1

Overview

Motivations

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before delivery?

Motivations

- Why is maintenance so effort-intensive?
- Why is progress measurement difficult?

Software

- *Instructions (computer programs) that when executed provide desired features, functions, and performance*

Software

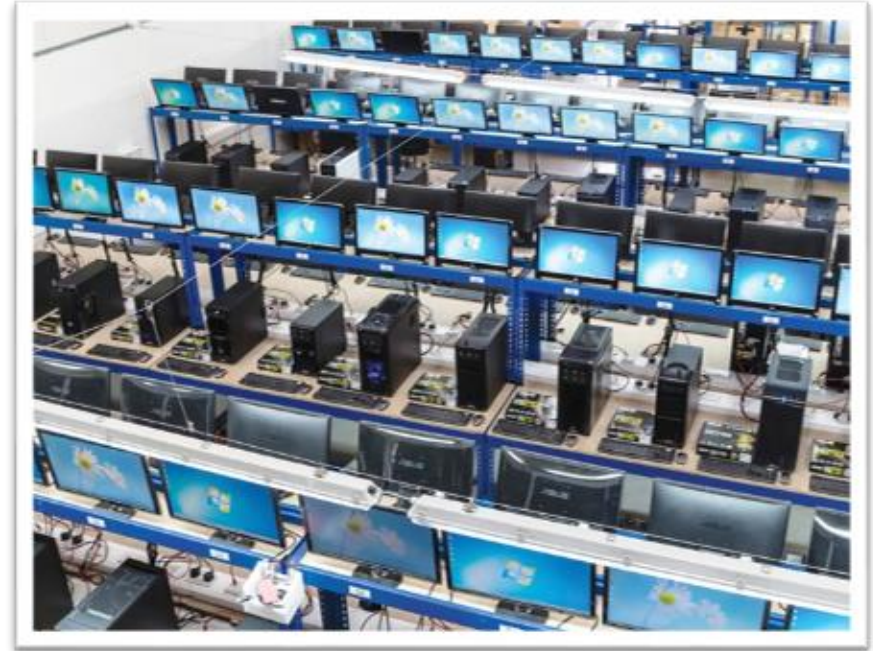
- *Data structures that enable the programs to adequately manipulate information*

Software

- *Descriptive information in both hard copy and virtual forms that describes the operation and use of the programs*

Software vs. Hardware

- Software is developed or engineered; it is not manufactured in the classical sense



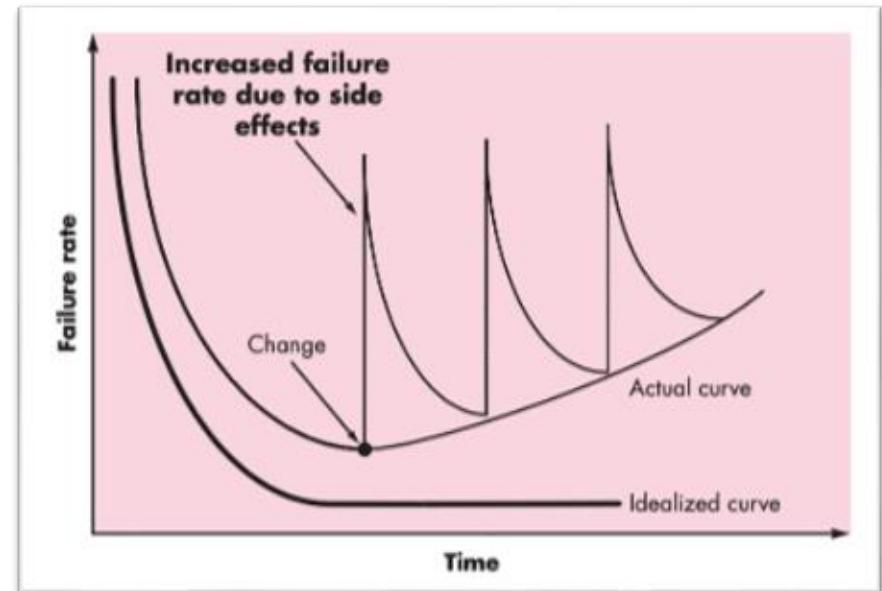
Computer Manufacturer

Software vs. Hardware

- Software doesn't "wear out"*



Hardware Lifetime



Software Lifetime

Software vs. Hardware

- Software Ages through:
 - dependency obsolescence
 - security vulnerabilities
 - technical debt
 - changing platforms
 - changing user requirements.

Software vs. Hardware

- While modern development increasingly relies on frameworks, libraries, services, and reusable components, significant customization is still required for many software systems.

Software Engineering (SE)

- *The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software. IEEE*

Software Engineering (SE)

- *The establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.* Fritz Bauer

SE Layers

- The SE approach to software development is often viewed using a layered architecture
- Layered architectures are easier to learn/implement, and each layer provides services to the ones above

SE Layers

- Quality
- Process
- Methods
- Tools

SE Layers

- Tools Layer:
 - Refers to automated or semi-automated tools that offer support for the processes and the methods a particular SE project is following

SE Layers

- Tools Layer:
 - Most common tools used involve project management, team communication, version control, code static analysis, automated testing, code coverage analysis, automated deployment, documentation, etc.

SE Layers

- Tools Layer:
 - Git and GitHub/GitLab
 - CI/CD Pipelines
 - Docker
 - Cloud Platforms
 - AI-assisted Development Tools

SE Layers

- *A method* is a particular way of doing something, like the specific techniques used



SE Layers

- In SE, a *method* refers to a specific approach to software development, usually based on a particular set of principles and practices
- Methods provide the technical how-to for building software

SE Layers

- Methods can refer to many different tasks that include communication, requirement analysis, design modeling, program construction, testing, and support

SE Layers

- A *methodology* (sometimes referred to as framework or model) is a set of methods used to achieve a goal

SE Layers

- In SE, a methodology refers to a specific approach to software development that is based on a particular set of principles and practices

SE Layers

- Examples of SE methodologies are SDLC, Waterfall, Spiral, RUP, Agile*, XP, Scrum, etc.

* Agile is an umbrella term to define a high-level set of principles for software development

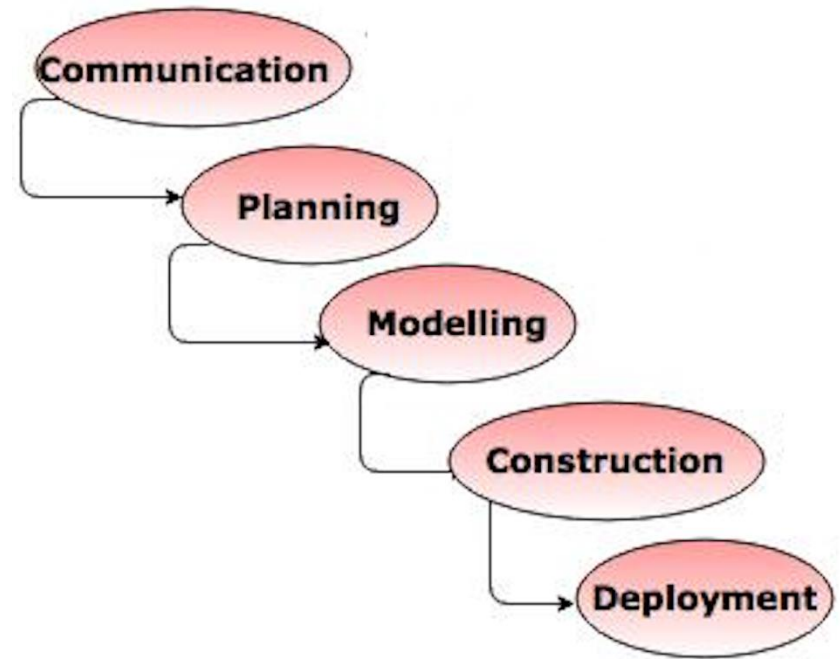
*

SE Layers

- A process describes in general terms the stages, phases, or milestones of a typical software development process
- In SE, a process refers to certain tasks that should be done within a timeframe

SE Layers

- Most SE processes can be described by the stages on the right



SE Layers

- Each stage in a software engineering process can be further refined into activities, actions, and tasks

SE Layers

- Tasks focus on a small, specific objectives
- Actions are a set of tasks that produce a major work product
- Activities are a group of related tasks and actions for a major objective

SE Layers

- For example, incorporate *authentication* can be an activity during a software development process

SE Layers

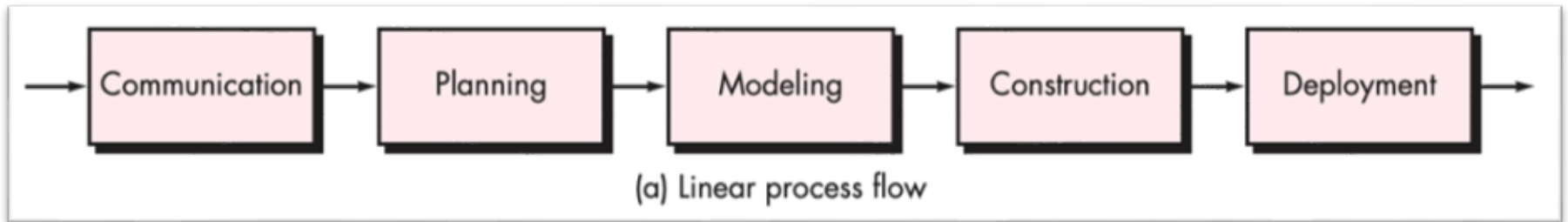
- Some of the actions relates to the *authentication* activity could be:
 - Decide which authentication mechanism to use and how user credentials will be stored
 - Implement and test the chosen authentication mechanism

SE Layers

- Within the process layer, a process flow describes how activities, actions and tasks that occur within each process stage are organized with respect to sequence and time

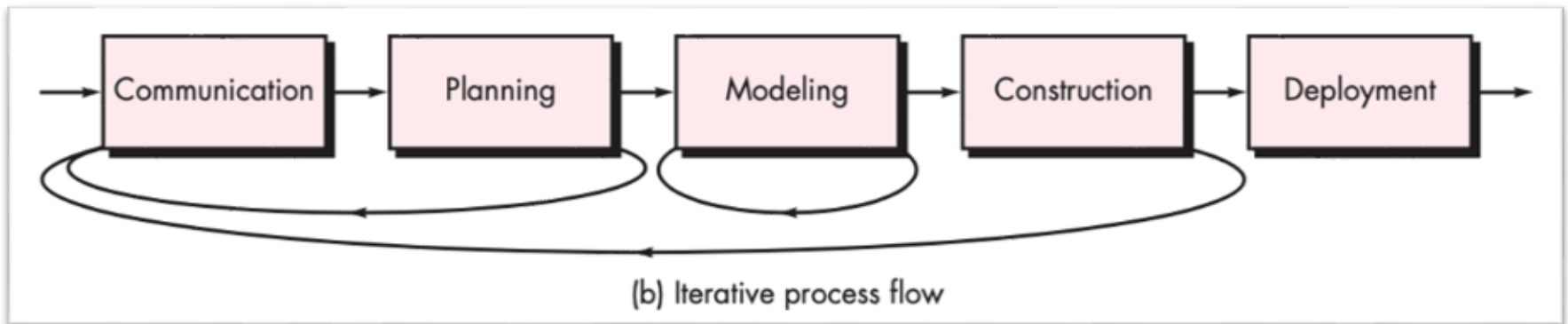
SE Layers

- Process Layer:



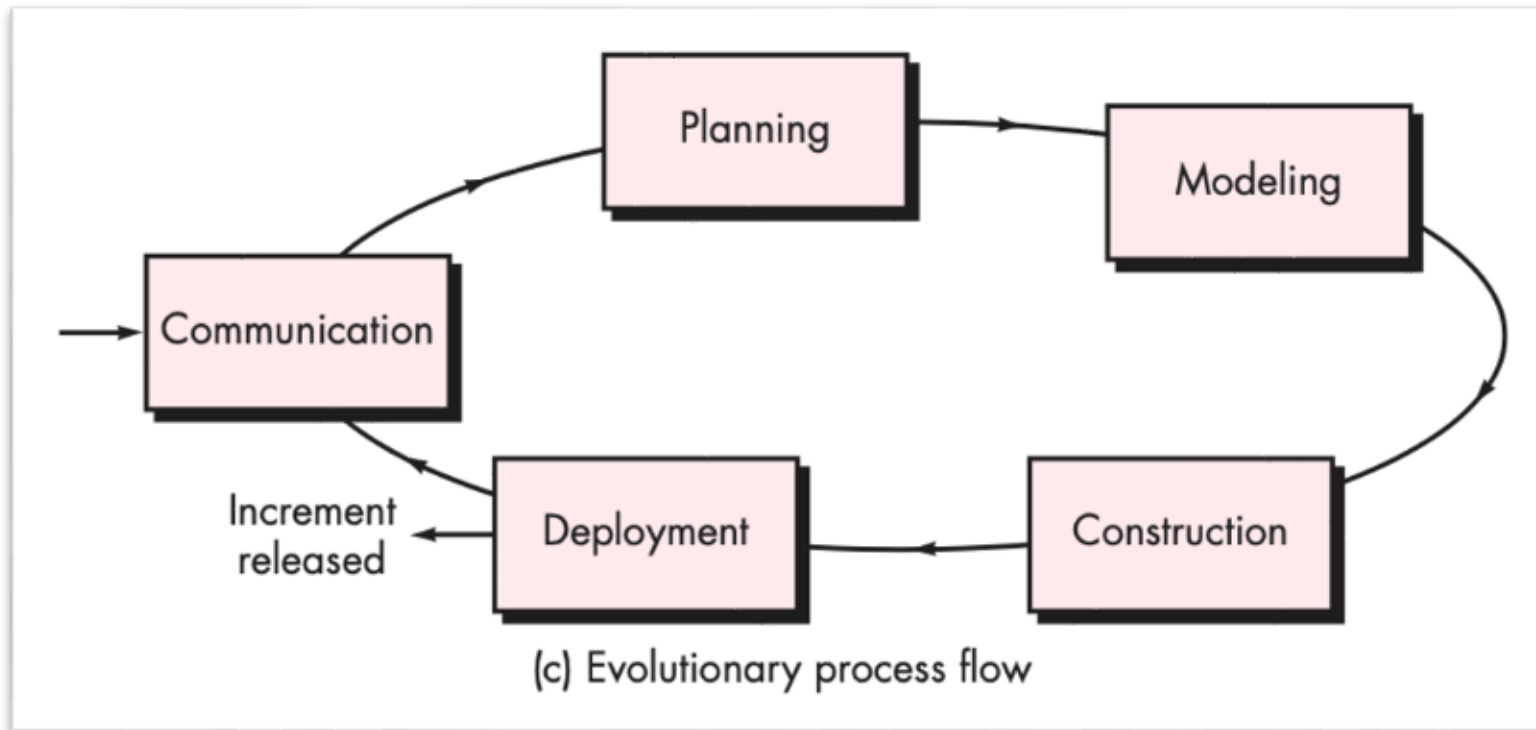
SE Layers

- Process Layer:



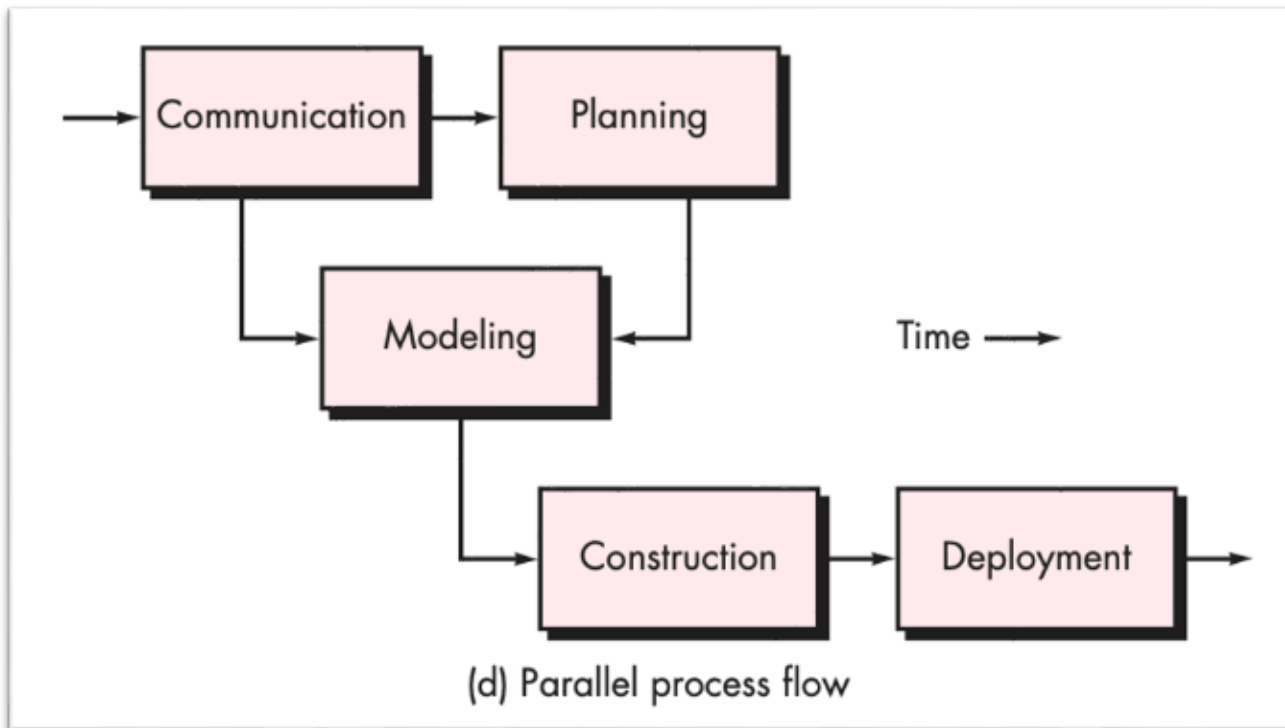
SE Layers

- Process Layer:



SE Layers

- Process Layer:



SE Layers

- Software quality can be defined as *an effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it*

SE Layers

- Software quality can be measured through a qualitative or quantitative scoring scheme (or a mix of both) and then a weighting system reflecting the priorities

SE Layers

- Testing is the basic way to measure all aspects of software quality
- The pressure to release software on time calls for the adoption of more software test automation

SE Layers

- Metrics commonly used are code quality, maintainability, usability, reliability, and security

DevOps

- A set of cultural principles, practices, and tools that integrates software development and operations to improve collaboration, automate workflows, and accelerate the delivery of high-quality software.

DevOps

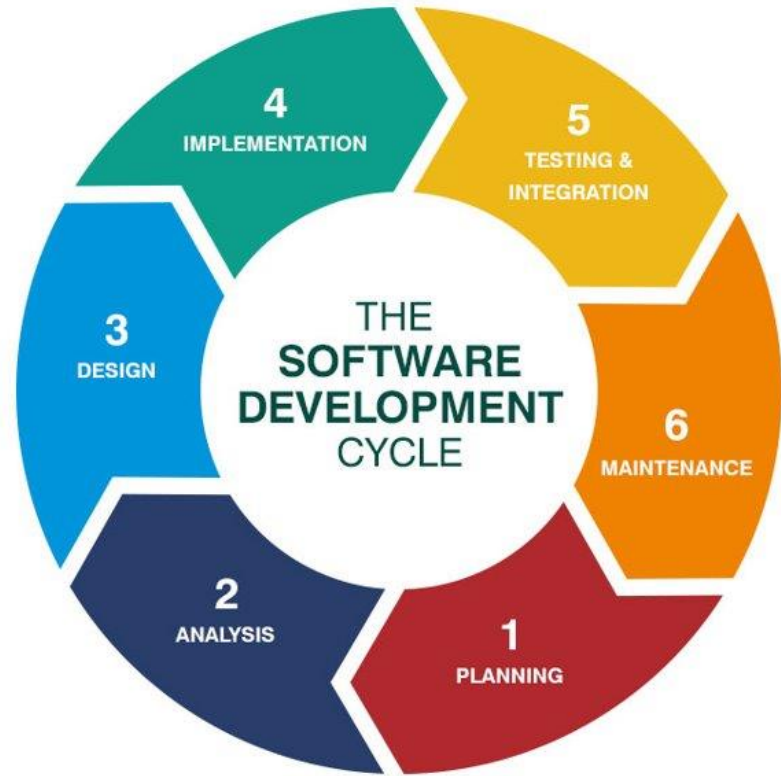
- It complements a software engineering process by focusing on how software is continuously built, tested, deployed, monitored, and operated.

SDLC

- Having explored the layered architecture of software engineering, we now turn to a foundational methodology called the Software Development Life Cycle (SDLC).

SDLC

- A software development methodology proposed in the 60s* and described by the phases on the side



SDLC: Planning

- Project plans are reviewed and approved by relevant stakeholders.
- A conceptual design of the system is conceived
- The project scope is defined
- Cost is estimated
- Schedule is drafted
- Resources are identified and allocated

SDLC: Planning

- What's the software's purpose?
- What problems does it solve?
- Who's the end user?
- What are the expectations?
- How much it will cost?
- How long it will take?
- Is it feasible?
- What are the risks?

SDLC: Analysis

- User and system requirements are specified
- User requirements describe features that the software provide to users and how well it supports those features
- They are written from the perspective of the users (or stakeholders)

SDLC: Analysis

- User requirements can be functional or non-functional
- Functional requirements describe what the software should do

SDLC: Analysis

- Examples of Functional Requirements:
 - users should be able to place lunch orders
 - users should be able to create recipes
 - students should be able to enroll in classes

SDLC: Analysis

- Examples of Functional Requirements:
 - users should be notified by email when a new order is created
 - users should have the ability to tag their recipes

SDLC: Analysis

- Non-functional requirements describe how well the software performs its functions
- Typically, they are expressed in terms of *performance*, *reliability*, *usability*, and *security* attributes

SDLC: Analysis

- Examples of Non-functional Requirements:
 - Users should get a response in less than one-tenth of a second (performance)
 - Users should be able to get a response 99.99% of the time or more (reliability)
 - Users should be able to access their recipes even when the connection is down (reliability)

SDLC: Analysis

- Examples of Non-functional Requirements:
 - Color-blinded users should be able to use the software (usability)
 - Spanish-speaking users should be able to use the software (usability)
 - Users should be able to search recipes using voice commands (usability)

SDLC: Analysis

- Examples of Non-functional Requirements:
 - Users should be able to authenticate themselves using unique IDs and passwords (security)
 - Users should be able to manage access permissions for their data (security)

SDLC: Analysis

- System requirements describe what the system should have to be able to run the software
- They focus on the technical characteristics and constraints that the system must meet to fulfill the user requirements

SDLC: Analysis

- Examples of System Requirements:
 - The software should run on Windows 11
 - The software should be able to run on Android version 7 and above
 - The software should not require more than 500MB of storage

SDLC: Analysis

- Examples of System Requirements:
 - The software should maintain all orders in persistent storage
 - The software should be able to authenticate users

SDLC: Design

- Requirements are turned into detailed specifications to be used by the software development team
- A more realistic plan is elaborated, including a timeline, budget, and resources

SDLC: Design

- System Architecture
- Database(s) and data model
- Programming language(s)
- Software development platform and tools
- APIs, libraries, frameworks
- User interface

SDLC: Implementation

- Includes not only the software that should be written, but also the infrastructure (hardware, software, communication system) that should be set in place

SDLC: Testing and Integration

- Checks whether the requirements specified in the design phase are being met
- Examples: unit testing, integration testing, user acceptance testing, security testing, etc.

SDLC: Maintenance

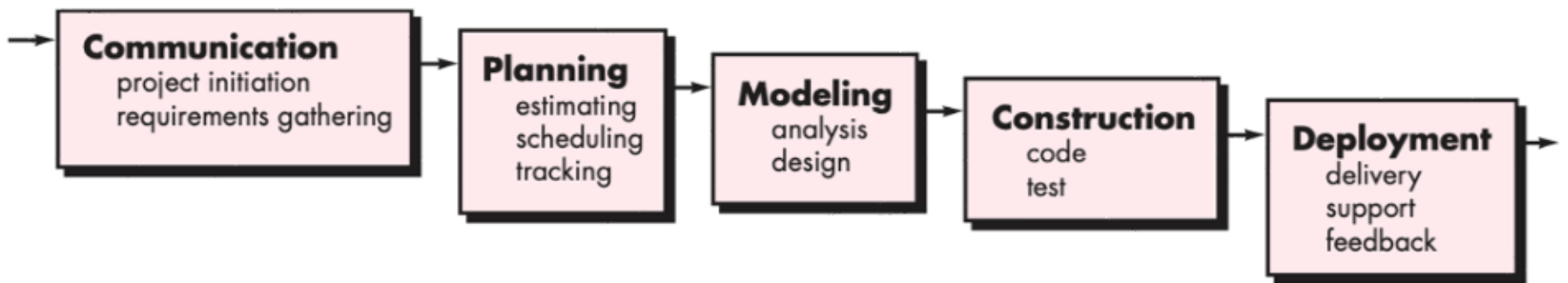
- The correction of problems identified after the system enters production
- It includes bug fixes, feature enhancements, security updates, performance improvements, dependency updates, cloud infrastructure changes, etc.

SDLC

- SDLC is a conceptual model from which other (somehow more specific) methodologies derived
- Example: Waterfall, Spiral, RUP, Agile, etc.

Waterfall

- Uses a linear process flow, which means that once you are done with a phase you normally don't go back (forward-only model, hence the analogy of a waterfall)



Waterfall

- Pros:
 - Simple and easy to understand
 - Easy to manage due to the rigidity of the model
 - Each phase has specific deliverables and a review process

Waterfall

- Pros:

- Phases are completed one at a time and do not overlap
- Works well for small projects where requirements are very well understood

Waterfall

- Cons:
 - Real projects rarely follow the sequential flow that the model proposes
 - It has difficulty accommodating the natural uncertainty that exists at the beginning of many projects

Waterfall

- Cons:
 - Customers must have patience: a working version of the program(s) will not be available until late
 - A major blunder, if undetected until the working program is reviewed, can be disastrous

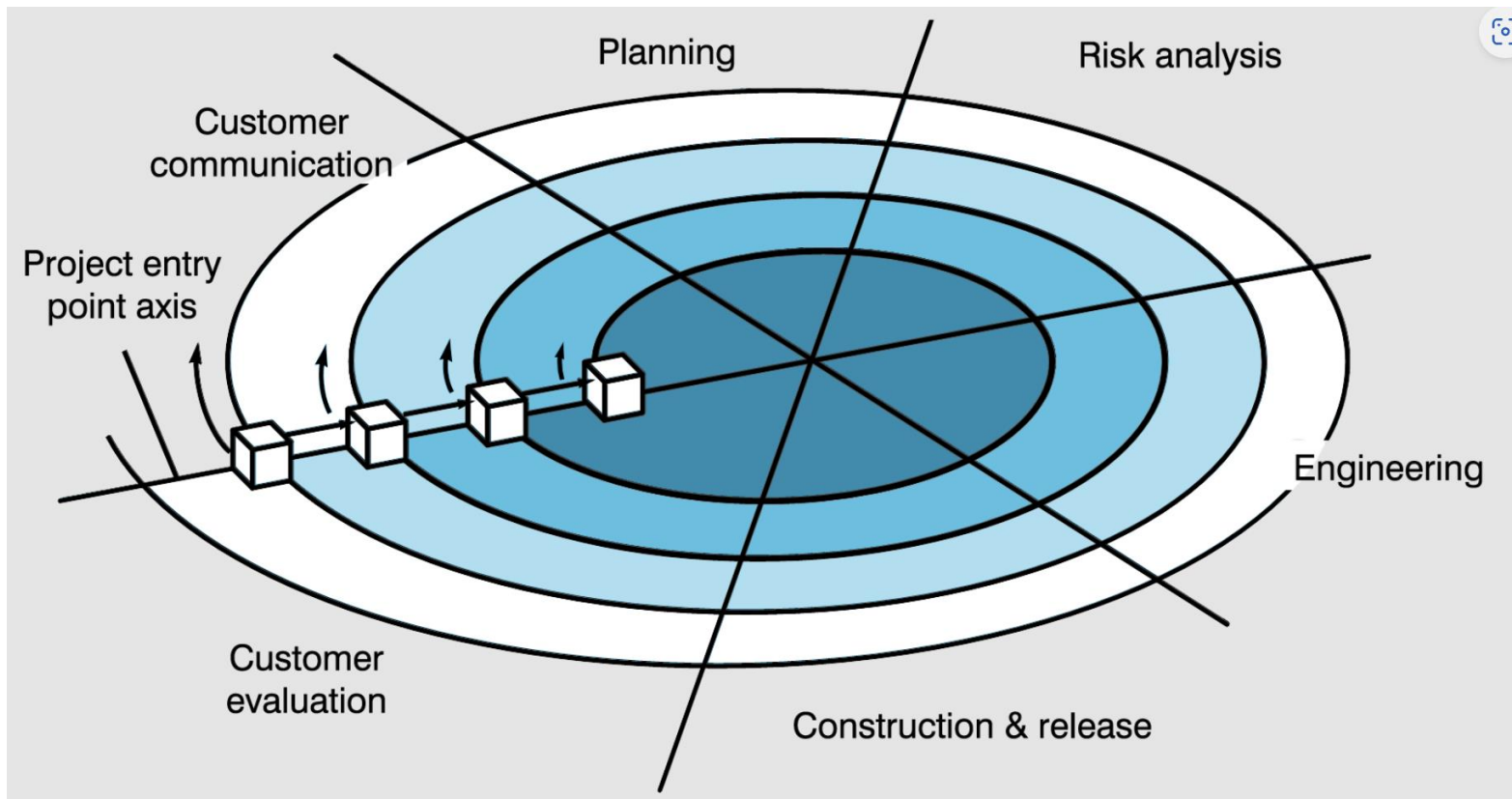
Waterfall

- When to use:
 - Objectives are clearly outlined from the beginning and the requirements are well understood

Spiral

- Provides a systematic and iterative approach to software development
- It is based on the idea of a spiral, with each iteration of the spiral representing a complete SDLC

Spiral



Spiral

- Pros:
 - Early software production
 - Great for risk handling
 - Flexibility in requirements

Spiral

- Cons:
 - Expensive for small projects
 - Complex to implement
 - Time management
 - Risk of spiral going on indefinitely*

Spiral

- When to use:
 - Large and complex projects
 - Higher risk projects
 - Need for stakeholder approvals as the project evolves

AI in Software Engineering

- Generative AI for coding assistance
- AI-assisted testing
- AI-assisted code review
- Prompt engineering
- Human oversight and software quality